

算子优化策略：从搬运到规约

目 录

1 引言：从 baseline 到高性能	2
2 让搬运与计算形成流水	2
2.1 Double Buffer	2
3 让中间结果留在 UB	2
3.1 复用 weight	2
4 根据 shape 设计 Tiling	3
4.1 为什么不一定需要启动全部 AIV	3
4.2 覆盖各种 shape	3
5 提高搬运效率	3
5.1 合并搬运	3
6 选择合适的规约实现与精度	4
6.1 规约实现	4
6.2 精度选择	4
7 拓展阅读	4
8 本章你将学会	4
9 要点速查	5
10 小结	5

1 引言：从 baseline 到高性能

有了编程模型和 profiling 工具，我们终于可以谈优化。本章给出的是**可能有效**的优化方向，而不是必须实现的清单。请始终以 profiling 结果和性能数据为准，不要照抄思路。

直觉 优化就像治病：本章给你的是“常见病种和常用药”，但具体开什么药，要看你算子的“体检报告”（profiling）。同一种药对不同体质可能有相反效果，所以每次改动后都要重新体检验证。

本章我们讨论五个优化方向：搬运与计算流水、中间结果留 UB、shape Tiling、搬运效率、规约实现与精度。

2 让搬运与计算形成流水

AIV 的 MTE2、Vector 和 MTE3 可以分别执行搬入、计算和搬出。由于这三个单元相互独立，如果能在同一时间重叠不同单元的执行，将能大幅掩盖时间，获得加速。Ascend C 的编程模型倡导把一个 tile 的处理拆成 **CopyIn, Compute, CopyOut** 三段式，明确生产者和消费者关系，并用 TQue 在阶段间传递 LocalTensor 的所有权与同步事件。这样的编写范式能让编译器更好地识别依赖关系，并**自动**开启流水。

2.1 Double Buffer

将队列的 BUFFER_NUM 设为 2（或更多），可以为相邻 tile 交叠提供空间：当一份数据参与 Vector 计算时，另一份数据才有机会同时搬入或搬出。这就是 **Double Buffering**（双缓冲）技术。

Double Buffer 并不总是保证自动加速。它是否生效还需要满足相邻阶段之间没有不必要的全局同步等条件。用 `msprof op simulator` 验证流水是否真的重叠，而不是想当然。

3 让中间结果留在 UB

融合算子的主要价值，是让前一阶段的输出直接成为后一阶段的 UB 输入，而不是每经过一步就写回 GM。对本算子而言， $R = x + \text{residual}$ 随后还要用于输出 residual_out、平方和规约以及计算 y 。只要 UB 容量允许，就应保留可复用的 R ，并把写回 residual_out 的副本交给输出队列，避免为后续计算再次读取 GM。

直觉 不妨把 UB 想成你面前的操作台，GM 想成远处的仓库。每去仓库取一次料都要走路、排队，很慢。把要反复用的料（ R ）一直放在操作台上，用完才送走，能省掉多次往返。

3.1 复用 weight

weight 在所有行上相同，也可以按核或按 tile 复用。把它放在 VECCALC 的 TBuf 中常驻，而不是每个 tile 都从 GM 重新搬入，能进一步减少搬运量。

与此同时，应统计所有常驻 *buffer* 与双缓冲队列的总占用，避免为了复用而挤压 *tile* 大小或导致 *UB* 超限。*UB* 只有 192 KiB，寸土寸金。

4 根据 shape 设计 Tiling

多核和单核 tiling 需要一起设计：

1. 运行时通过平台接口查询 AIV 核数与 *UB* 容量，不把某个环境中的数值写死到策略中。
2. 各行相互独立时，优先沿独立维（如 *B*）切分。使用商和余数分配大小核，使各核行数最多相差 1，避免简单向上取整造成尾部空闲核。
3. 当独立维很小时，盲目启动全部 AIV 没有收益。沿其他维（如 *H*）切分虽然能增加并行度，却会引入跨核规约或额外 pass，需要根据实测决定是否值得。
4. *tile* 大小既要满足 *UB* 预算，也要让每核有足够多的 *tile* 支撑流水。最大的 *tile* 不一定最快，最小的 *tile* 也会增加循环和搬运启动开销。

4.1 为什么不一定需要启动全部 AIV

在 NPU 中，每个核在被启动时都要单独付一次**初始化开销**（加载配置、建立执行上下文、准备片上资源等），且这部分几乎无法通过流水、搬运或规约优化消除。它只和“启动了多少个核”有关。实测中，空 kernel 的 Task Duration 随启动核数近似线性增长（约 2 核 0.67 μ s $\rightarrow$ 40 核 2.25 μ s），本身处在微秒量级。

例 设算子整体只有 5 μ s，单核计算量又很小。若启动 40 个 AIV，光启动开销就约 2.25 μ s，占比近一半。此时用 10 个核可能就够了，启动开销降到约 1 μ s，反而更快。只有当算子整体耗时数十微秒以上、单核计算量大时，启动开销才被淹没，才值得写满全部 AIV。

4.2 覆盖各种 shape

公开评测 shape 中既有 $B = 1$ ，也有 *H* 非 32B 对齐的情况（如 $H = 3037$ ）。一个只对 [256, 1024] 快、但在小 *B* 或尾块上出错的 tiling 不是有效实现。你的 tiling 策略应能自适应各种 shape。

5 提高搬运效率

MTE 搬运存在固定启动成本，单次搬运过小时难以充分利用带宽。实测表明，HBM $\rightarrow$ UB 和 UB $\rightarrow$ HBM 的单核带宽随单次搬运数据量增长而上升，小搬运无法跑满带宽。

直觉 不妨把搬运想成物流车：派一辆大卡车拉一整车货，比派十辆小面包车各拉一点要高效得多。但卡车太大也会装不满、空跑，要和货物量（*tile* 大小）匹配。

5.1 合并搬运

若 *UB* 预算和数据布局允许，可以尝试用一次 DataCopy 搬运多行，或用 blockCount、blockLen 与 stride 参数表达规则的多段搬运，减少逐行发射指令的开销。与此同时需要注意：

1. GM 起始地址、每行 stride 和 UB 地址的对齐会共同影响搬运效率。
2. 非对齐尾块应使用 DataCopyPad 或等价 mask 正确处理，不能为了对齐越界读写。
3. 合并搬运会增加 UB 占用，并可能减少每核 tile 数，应与 Double Buffer 一起重新评估。

6 选择合适的规约实现与精度

6.1 规约实现

高效的归约操作是本算子获得高性能的关键一环。Ascend C API 中提供的 Reduce<op> 类算子可能存在较多的同步 flag 设置和边界检测情况，实际上可能并不能获得很好的性能。因此，Ascend C API 同时提供了更加底层的 BlockReduce<op> 操作和 WholeReduce<op> 操作。两者在单次执行速度和吞吐效率上各有优劣，但合作起来就可能获得比原生 Reduce<op> 更好的规约性能和运算单元利用率。

这部分是本实验最能拉开差距的优化点之一。建议先用 Reduce<op> 跑通正确性，再用 BlockReduce/WholeReduce 替换，每次替换后用 profiling 对比 Vector 利用率和端到端耗时。

6.2 精度选择

本实验的输入输出是 FP16，但这并不意味着平方和也适合在 FP16 中累加。使用 FP32 作为中间结果通常更稳健，但会占用更多 UB，也可能增加 Cast 和 Vector 计算开销。可以尝试减少重复 Cast、缩短 FP32 数据的存活范围。

例 取 $H = 1024$ 、FP16 输入来看为什么中间用 FP32 更稳。FP16 的有效尾数约 11 bit，最大可精确表示的整数约 2048。若 1024 个 FP16 平方值（范围 $[0, 1]$ ）在 FP16 下累加，累积舍入误差可能不可忽略；改用 FP32（有效尾数约 24 bit）累加，再转回 FP16 输出，精度更有保障。代价是多两次 Cast（FP16→FP32, FP32→FP16）和额外的 FP32 UB 占用。

禁止一味地使用低精度来获得更高性能。任何低精度或近似方案都必须通过最后的精度校验。

7 拓展阅读

昇腾社区提供了一系列 Ascend C 算子性能优化实用技巧文章，涵盖流水优化、内存优化、搬运优化、Tiling 优化和 API 使用优化，是深入理解本算子优化的好材料。实际编写时建议对照阅读。

8 本章你将学会

1. 用三段式（CopyIn/Compute/CopyOut）和 BUFFER_NUM=2 开启搬运与计算的 double buffer 流水，并用 simulator 验证流水是否重叠。
2. 让中间结果 R 留在 UB，复用 weight，同时管理 UB 总占用避免超限。

3. 根据 shape 设计多核与单核 tiling，用商余分配均衡各核，判断是否值得启动全部 AIV。
4. 合并搬运提高 MTE 带宽利用率，用 DataCopyPad 处理非对齐尾块。
5. 用 BlockReduce/WholeReduce 替换 Reduce<op> 提升规约性能，在 FP16 输入下用 FP32 累加保证精度。

9 要点速查

优化方向	核心手段	验证指标
搬运计算流水	三段式 + BUFFER_NUM=2 double buffer	simulator 看 Vector 空泡减少
UB 复用	R 留 UB, weight 常驻 TBuf	MTE2/MTE3 带宽下降, UB 占用不超限
Tiling	沿 B 切分, 商余均衡, 自适应核数	各核工作量均衡, 无明显拖尾
启动开销	算子小则少用核, 大则写满	Task Duration vs 启动核数
搬运效率	合并搬运, blockCount/stride	单次搬运带宽利用率上升
尾块	DataCopyPad 或 mask	非 32B 对齐 shape 正确
规约实现	BlockReduce+WholeReduce 替代 Reduce	Vector 利用率, 规约耗时
精度	FP16 输入, FP32 累加, 减少 Cast	精度校验通过

10 小结

本章从 profiling 发现的瓶颈出发，讨论了五个优化方向：用三段式和 double buffer 让搬运与计算流水重叠；让中间结果 R 和 weight 留在 UB 减少 GM 往返；根据 shape 设计多核与单核 tiling，权衡启动开销与并行度；合并搬运提高 MTE 带宽利用率，用 DataCopyPad 处理尾块；用 BlockReduce/WholeReduce 提升规约性能，在 FP16 输入下用 FP32 累加保证精度。每个方向都应以 profiling 证据为依据，一次改一个变量，验证正确性后再比较耗时。下一章我们将把构建运行、评分与提交的具体操作讲清楚。

讲义基于 HPC101 2025 Lab3.5 实验内容编写